In order to reprise for the purpose of re-orientation (for the reader), the purpose of my Capstone project is to create good conversations about mental-math arithmetic (non-algorithmically-solved, solved-only-in-head arithmetic), even when students get answers to arithmetic answers wrong, or more precisely *especially when* students get answers to arithmetic answers wrong. In that vein, a method was needed in order to *score* the performance of my math bot according to how well the bot creates good conversational output relevant to a math problem, especially when a student answers *incorrectly* to the problem.

11 JSON files were created with multiple arithmetic problems building up to a last problem in each file, where the previous problems in each file could help a student solve the last problem.

What has become a canonical problem-set is:

4 * 25 = 100
4 * 250 = 1000
1000 / 4 = 250
996 / 4 = 249

… where the solution to “996 / 4,” without using long division, can be mentally reasoned out (without writing anything down) using the answers to the previous, easier problems “4 * 25,” “4 * 250,” and “1000 / 4” and then the distributive math property.

Basically, each of the problems in that series of arithmetic problems allows the next problem to be solved more easily, adding incremental knowledge.

Because the job of my math bot is to create productive arithmetic math conversation, I have setup a Python notebook to do the following:

1. Only ask the last question in the problem set, which in the case above is 996 / 4. Also ask the student for reasoning for their answer.
2. Submit a pretend-student-answer “dunno,” with pretend reasoning “dunno.”
3. Have the math bot give another related question, to try to help the student get past “dunno” (basically lost entirely) for both an answer and reasoning.
4. Have Gemini evaluate the relevance of the math bot’s answer to important topics:
    a) The student’s answer from step 2,
    b) The correct answer to the problem (249 in the specific case above), and
    c) A list of related math properties that can help the student solve the problem correctly (the distributive math property, in the case above), which has been previously written in the JSON input file along with the math problem set.

5. Repeat step 1.
6. Now submit a pretend-student answer with “dunno,” but now with pretend reasoning constructed of all the questions-and-answers to previous problems in the problem set,

plus the related math properties given by the teacher in the JSON input file for the last problem (the distributive math property, for this example).

7. Repeat step 4.

The Gemini evaluations in steps 4 and 7 are given in ratings of 0 to 10, where for instance for 4(c) above, a rating of 0 signifies that the math bot answer and the math properties given in the JSON file are totally unrelated, and 10 signifies that the math bot answer and the math properties given in the JSON file are extremely related. The scoring is made on the 3 “axes” from 4 (a), (b), and (c) and each score in the range of 0 to 10 is normalized to the range of 0 to 1. The scores are then made into 3-element vectors. So for instance, a math chatbot answer that Gemini considers to have “perfect relevant” to a student’s reasoning, the correct answer to the problem, and to related math properties given in the JSON file, is given a vector score of [1, 1, 1], whereas a math chatbot answer that Gemini considers to have “no relevance” to a student’s reason, to the correct answer to the problem, and to related math properties is awarded a vector score of [0, 0, 0].

The Gemini scoring described above was done on the last problem in each of the 11 problem sets listed here:

25 * 4 =
250 * 4 =
1000 / 4 =
996 / 4 =

---

10 *15 =
20 * 15 =
2 * 15 =
1 * 15 =
495 / 15 =

Find Fractions to Match the Following
Decimal Numbers:
0.333… =
0.111… =
0.222… =
0.666… =
0.999… =

---

10 * 0.333… =
3.333… - 0.333… =
1000 * 0. 4858585… =
10 * 0.4858585… =
485.858585… - 4.858585… =

---

108 − 12 =
12 − 108 =
108 − 22 =
22 − 108 =
183 − 54 =
54 − 183 =
21,054 − 21,183 =

Please answer as decimal answers
rounded to the nearest thousandths:
1 / 3 =
1/6 =
5/6 =
1/12 =
11/12 =

7×8 =
14 × 4 =
7×9 =
21 × 3 =
7 × 17 =

Please answer as exact decimal answers,
and then also numbers rounded to the
nearest tenth’s place:
720 ÷ 960 =
360 ÷ 480 =
36 ÷ 48 =
33 ÷ 44 =
3 ÷ 4 =

---

4 * 4=
400 * 40 =
3 * 4=
30 * 40 =
431 * 40 =

---

5 * 20 =
50 * 20 =
1000 / 20 =
980 / 20 =
990 / 20 =

---

3 * 2 =
7 * 4 =
14 * 2 =
0.14 * 2 =
3.14 * 2 =
3.14 * 0. 2 =
(314 / 1000) * (2 / 10) =

After getting score vectors for the last problem in each of these problem sets, my evaluation method adds all the score vectors together (using standard vector addition), and then finds the *distance* of the tip for the score-vector-summation to the tip of the “perfect-score-vector” which is constructed of a summation of a [1, 1, 1] vector for each problem. That distance is then divided by the length of the “perfect-score-vector” (the norm of the “perfect-score-vector”), which results in a fraction between 0 and 1 which can be converted to a percentage. For a “control run” where 11 identical problems were used, where the chat bot was provided abundantly relevant “related properties” to the problem, and where the student always gave a correct answer but embedded the answer in the phrase “Something around like [correct_answer_from_JSON_file] but not exactly ?" … a score of 87.1% was attained for this control-run by my chat bot.

For a non-control run where the 11 problem-sets above were used, my math bot attained a score of 67.8% as of Oct 19, 2026.